

Experiment No :- 1

NAME :

DATE : / /

4-bit Ripple Carry Adder

Diagram :-

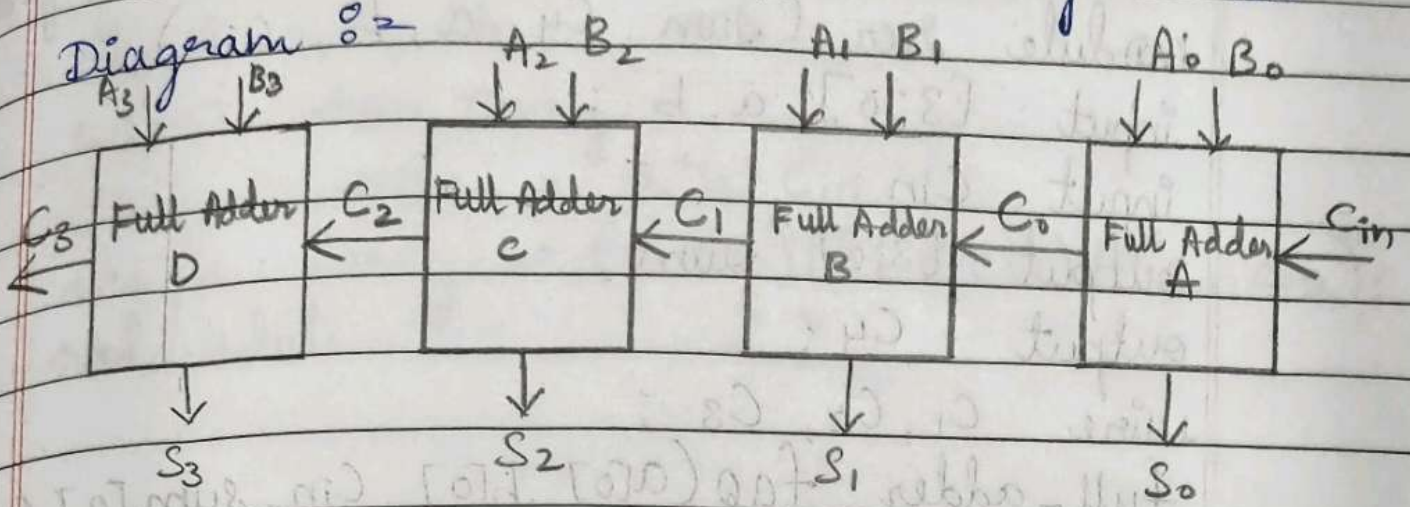


fig:- 4-bit Ripple Carry Adder

Truth Table :-

A	B	C_{in}	S	C_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

program

begin

```
module rca (sum, c4, a, b, cin);
```

```
input [3:0] a, b;
```

```
input cin;
```

```
output [3:0] sum;
```

```
output c4;
```

```
wire c1, c2, c3;
```

```
full_adder fa0 (a[0], b[0], cin, sum[0], c1);
```

```
full_adder fa1 (a[1], b[1], c1, sum[1], c2);
```

```
full_adder fa2 (a[2], b[2], c2, sum[2], c3);
```

```
full_adder fa3 (a[3], b[3], c3, sum[3], c4);
```

```
endmodule
```

=> Test Bench

```
module tb_v;
```

```
reg a;
```

```
reg b;
```

```
reg cin;
```

```
cin = 0;
```

```
a = 4'b0110;
```

```
b = 4'b1100;
```

```
#10;
```

```
a = 4'b0111;
```

```
b = 4'b1110;
```

```
#10;
```

```
end
```

```
endmodule
```

```
a = 4'b0010;
```

```
b = 4'b0100;
```

```
cin = 1;
```

```
#100;
```

```
endmodule
```

=>

```
module fulladder (a, b, cin, sum, carry);
```

```
input a, b, cin;
```

```
output sum, carry;
```

```
assign sum = a ^ b ^ cin;
```

```
assign carry = (a & b) | (b & cin) | (cin & a);
```

```
endmodule
```

Result: d s f d s f d

	0ms	200	400ns	500
New				
sum[3:0]	4'b0111	4'b0111	4'b0111	
cin	1			
a[3:0]	4'b0010	4'b0010	4'b0010	
b[3:0]	4'b0100	4'b0100	4'b0100	
c4	0	0	0	

Verilog code

```

module yfqwant (clk, rst, out);
input clk, rst;
output reg [3:0] out;
always @ (posedge clk, posedge rst)
if (rst)

```

```

out <= 4'hf;

```

```

else
out <= {out[2:0], (out[3] ^ out[2])};
endmodule

```

Test Bench

```

module stftb_v;
reg clk;
reg rst;
wire [3:0] out;
yfqwant uut (
.clk (clk),
.rst (rst),
.out (out)
);

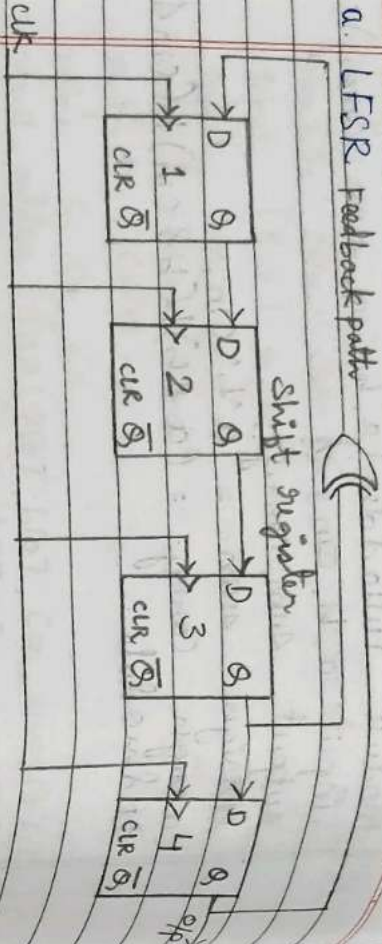
```

```

initial begin
clk = 0;
rst = 1;

```

a. LFSR Feedback path



Circuit diagram



#50;

out = 0;

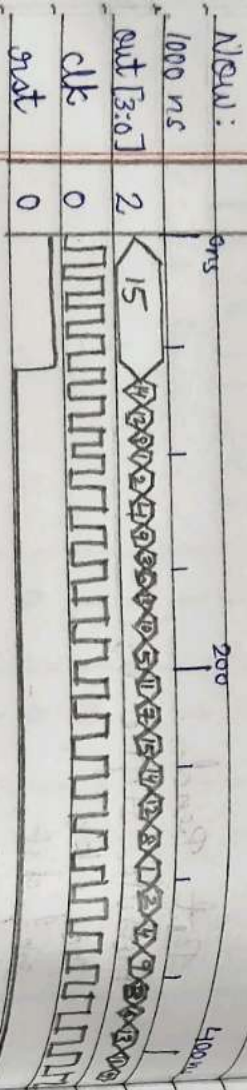
#50;

end

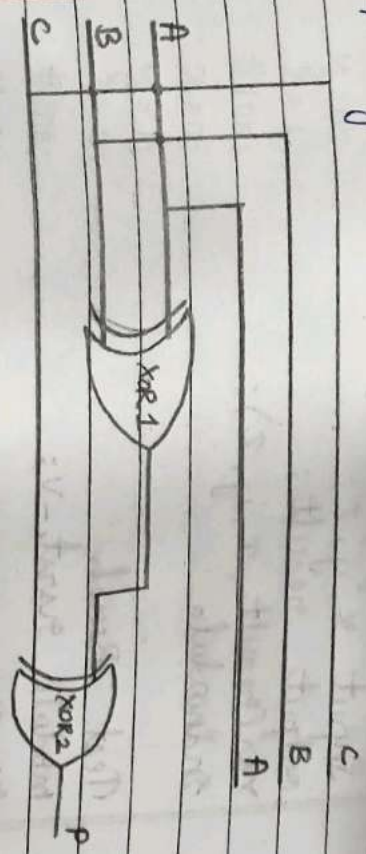
always #5 clk = ~clk;

endmodule

Result :-



b. Parity Generator :-



Truth Table :-

A	B	C	P
0	0	0	1
0	0	1	0
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1

Verilog code

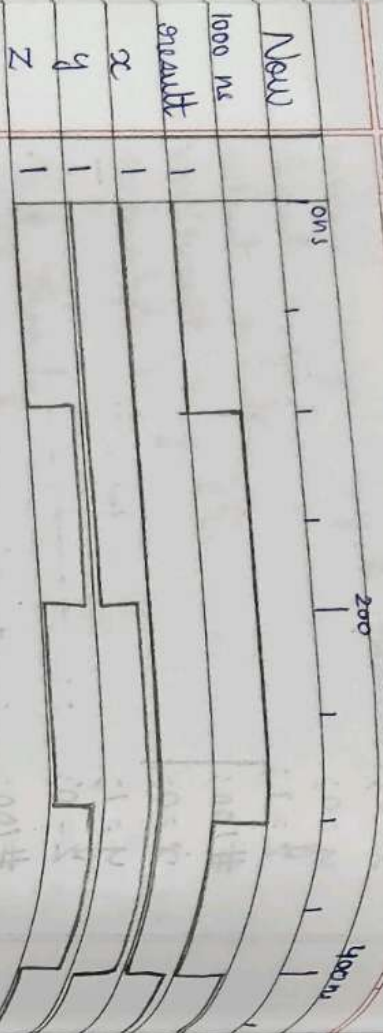
```
module wdgpr (x, y, z, result);
    input x, y, z;
    output result;
    xor(result, x, y, z);
endmodule
```

Test Bench

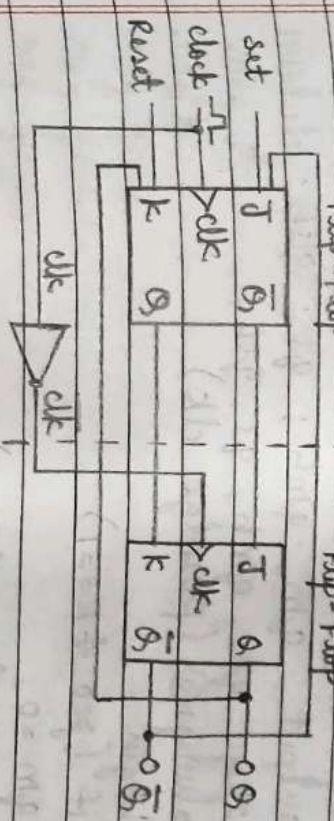
```
module test_v;
    reg x;
    reg y;
    reg z;
    wire result;
    wdgpr uut(
        .x(x),
        .y(y),
        .z(z),
        .result(result)
    );
    initial begin
        x = 0;
        y = 0;
        z = 0;
        #100;
    end
```

```
x = 0;
y = 0;
z = 1;
#100;
x = 0;
y = 1;
z = 0;
#100;
x = 1;
y = 0;
z = 0;
#100;
x = 1;
y = 1;
z = 1;
#100;
end
endmodule
```


Result :-



Master Slave JK Flip Flop :-



clk	J	K	Q _n	Master flip-flop o/p	Slave flip-flop o/p (Q _{n+1})
↑	0	0	0	0	No change
↓	0	0	0	No change	0
↑	0	0	1	1	No change
↓	0	0	1	No change	No change
↑	0	0	0	No change	0
↓	0	0	0	No change	No change
↑	0	0	1	1	No change
↓	0	0	1	No change	0
↑	1	1	0	1	No change
↓	1	1	0	No change	1
↑	1	1	1	1	No change
↓	1	1	1	No change	No change
↑	1	1	0	1	No change
↓	1	1	0	No change	0

module msk (j, k, clk, qm, qmb, qs, qsb);

input j, k, clk;

output qm, qmb, qs, qsb;

reg qm, qmb, qs, qsb;

always @ (posedge clk)

begin

if (j=0 & k=1)

qm = 0;

else if (j=1 & k=0)

qm = 1;

else if (j=0 & k=0)

qm = qm;

else if (j=1 & k=1)

qm = ~qm;

qmb = ~qm;

end

always @ (negedge clk)

begin

qs = qm;

qsb = ~qs;

end

endmodule

=> Test Bench

module statat_v;

reg J;

reg k;

reg qm;

reg qmb;

reg qs;

reg qsb;

wire result;

always with (

initial

J(0),

k(0),

qm(qm),

qmb(qmb),

qs(qs),

qsb(qsb)

);

always #10 clk = ~clk;

initial begin

J=0;

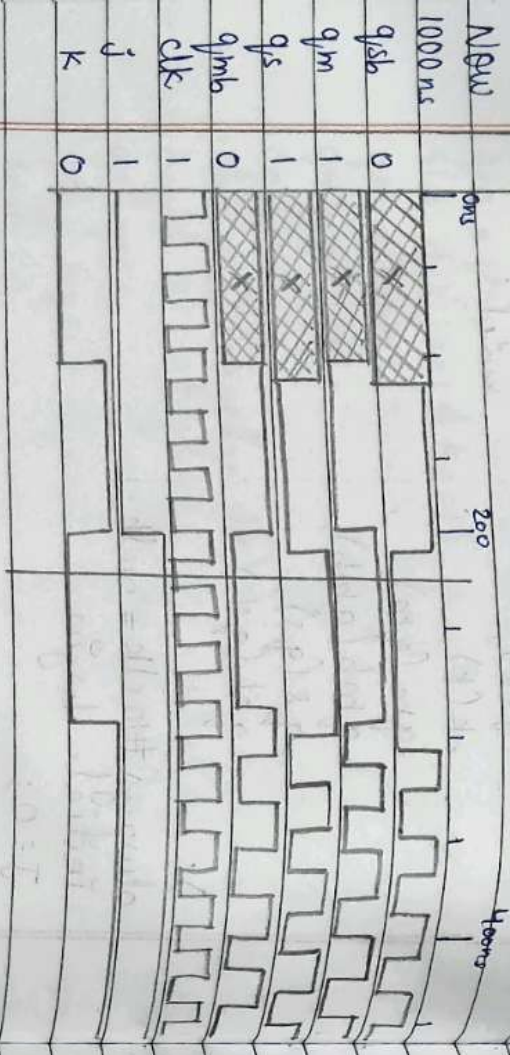
k=0;

clk=1;

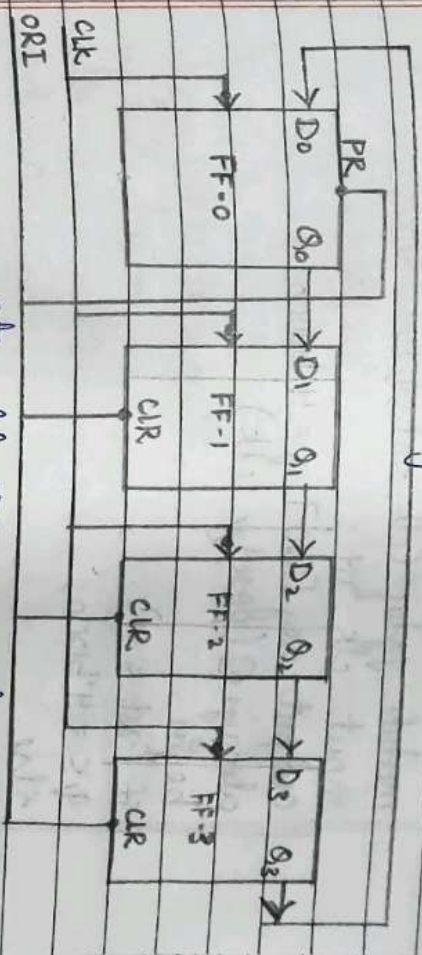
#100;

$J=0;$
 $K=1;$
 $clk=1;$
 $\#100;$
 $J=1;$
 $K=0;$
 $\#100;$
 $J=1;$
 $K=1;$
 $\#100;$
 $\$end$
 synchmodule

Result :-



4-Bit Ring Counter



Straight Ring Counter

Truth Table

ORI	clk	Q ₀	Q ₁	Q ₂	Q ₃
low	X	1	0	0	0
high	low	0	1	0	0
high	low	0	0	1	0
high	low	0	0	0	1
high	low	1	0	0	0

presented 1

Verilog code

```

module jgduf (clk, rst, q);
input clk, rst;
output reg [3:0] q;
always @(posedge clk)
begin
if (rst == 1)
q <= 4'b1000;
else
begin
q[3] <= q[0];
q[2] <= q[3];
q[1] <= q[2];
q[0] <= q[1];
end
end
endmodule

```

Test Bench

```

module softgh-v;
reg clk;
reg rst;
wire [3:0] q;

```

```

gfd5 uut (
.clk(clk),
.rst(rst),
.q(q)
);

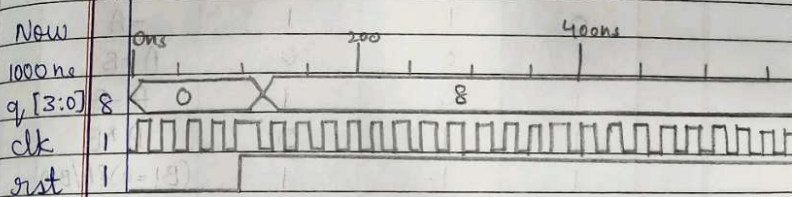
```

```

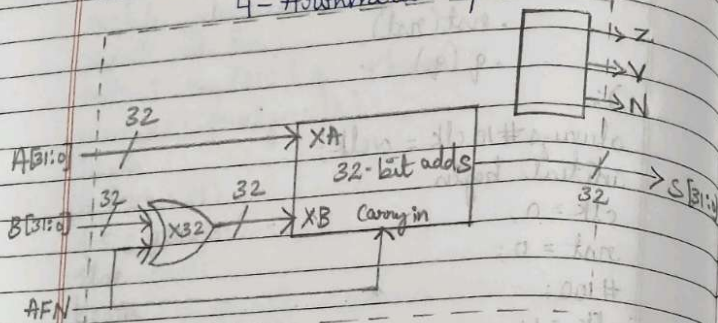
always #10 clk = ~clk;
initial begin
clk = 0;
rst = 0;
#100;
clk = 1;
rst = 1;
#100;
end
endmodule

```

Result :-



32-Bit ALU Supporting 4-Logical and 4-Arithmetic Operations



Truth Table 8=

INPUT 1	INPUT 2	INPUT 3	OPERATIONS
0	0	0	$\sim A$
0	0	1	A/B
0	1	0	$A \& B$
0	1	1	$-A$
1	0	0	$A+B$
1	0	1	$A-B$
1	1	0	$A*B$
1	1	1	$(B \neq 0) ? A/B : 0$

Verilog code

```

module alu (A, B, ALU_sel, ALU_out);
    input [31:0] A, B;
    input [2:0] ALU_sel;
    output reg [31:0] ALU_out;
    always @(*) begin
        case (ALU_sel)
            3'b000: ALU_out = ~A;
            3'b001: ALU_out = A/B;
            3'b010: ALU_out = A & B;
            3'b011: ALU_out = -A;
            3'b100: ALU_out = A+B;
            3'b101: ALU_out = A-B;
            3'b110: ALU_out = A*B;
            default: ALU_out = 32'b0;
        endcase
    end
endmodule

```

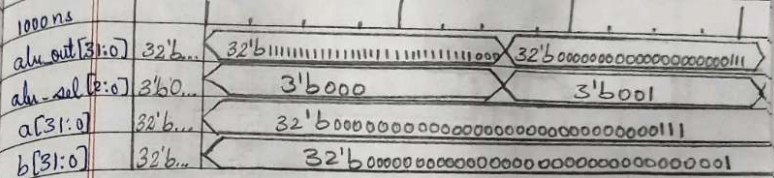

NAME: _____
DATE: 1/1

);
initial begin

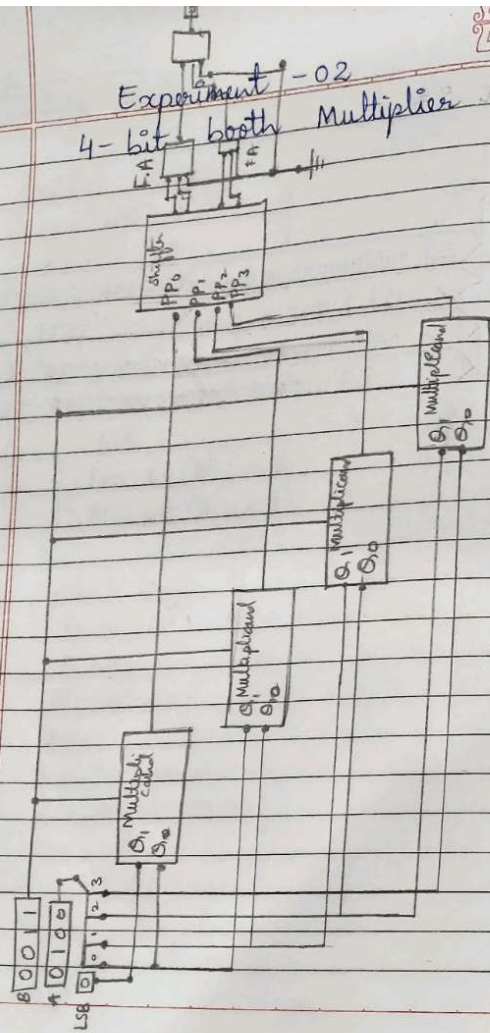
#100;
AW_sel = 3'b001;
#10;
LW_sel = 3'b010;

NAME: _____
DATE: / /

Now:-



Experiment - 02 4-bit booth Multiplier



Truth - Table

bi+1	bi	bi-1	val	x1 a	x2 b	Z	Neg
0	0	0	0	1	0	1	0
0	0	1	1	0	1	1	0
0	1	0	1	0	1	0	0
0	1	1	2	1	0	0	0
1	0	0	-2	1	0	0	1
1	0	1	-1	0	1	0	1
1	1	0	-1	0	1	1	1
1	1	1	0	1	0	1	1

=> verilog code

```

module boothmulti(x,y,z);
input signed [3:0] x,y;
output signed [7:0] Z;
reg signed [7:0] z;
reg [1:0] temp;
integer i;
reg E1;
reg [3:0] y1;
always @(x,y)
begin
Z = 8'd0;
E1 = 1'd0;

```


DATE: / /

```

for (i=0; i<4; i=i+1)
begin
temp = {x[i], E1};
y1 = -y;
case (temp)
2'd2: z[7:4] = z[7:4] + y1;
2'd1: z[7:4] = z[7:4] + y;
default: begin end
endcase
z = z >> 1;
z[7] = z[6];
E1 = x[i];
end
if (y == 4'd8)
begin
z = -z;
end
end
endmodule

```

NAME: / /

Test Bench :=

```

module boot_v;
reg [3:0] x;
reg [3:0] y;
wire [7:0] z;
boot uut (
    .x(x),
    .y(y),
    .z(z),
);
initial begin
x = 0000;
y = 0000;
#100;
x = 0001;
y = 0011;
#100;
x = 1001;
y = 0011;
#100;
x = 1001;
y = 1011;
#100;

```

```

x = 0101;
y = 1010;
#100;
end
endmodule

```

Result :-

